

ACTIVIDAD 1 — Diagnóstico del código

Objetivo: identificar al menos cinco aspectos de calidad que podrían mejorarse sin modificar el código.

N.º	Problema	Ubicación	¿Por qué debería mejorarse?
1	Componente de interfaz demasiado extenso y con responsabilidades mezcladas.	src/app/dashboard/usuarios/page.tsx (537 líneas; lógica de carga, validación, vínculos padre-hijo, edición y renderizado en el mismo componente).	Dificulta leer, probar y modificar el módulo. Conviene separar formulario, tabla/listado, edición de relaciones y lógica de datos en componentes/hooks más pequeños.
2	Validaciones duplicadas.	src/app/dashboard/usuarios/page.tsx y src/lib/validations.ts; además guardarEdicion vuelve a validar nombre, apellido y DNI manualmente.	La misma regla aparece en varios lugares y puede quedar inconsistente. Una única fuente de validación reduce errores y mantenimiento duplicado.
3	Uso excesivo de any y desactivación global de la regla no-explicit-any.	Varios archivos de src/services, por ejemplo actividades.service.ts, asistencias.service.ts, inscripciones.service.ts, perfiles.service.ts y roles.service.ts.	Se pierde gran parte de la seguridad de TypeScript. Los errores de forma de datos pasan a detectarse en ejecución en vez de durante compilación.
4	Consulta N+1 al calcular cupos de actividades.	src/services/actividades.service.ts → obtenerActividadesConCupos(): primero obtiene actividades y luego ejecuta una consulta count por cada actividad.	A medida que aumenten las actividades, crece la cantidad de consultas. Conviene obtener los conteos de forma agregada mediante una vista, RPC o consulta agrupada.
5	Documentación README incompleta respecto de la consigna de clase.	README.md del repositorio.	Incluye descripción, requisitos y ejecución, pero no presenta claramente las funcionalidades principales ni los integrantes del equipo, ítems pedidos por la actividad.

Pregunta de diagnóstico

¿El programa funciona correctamente? ¿Significa esto que necesariamente está bien desarrollado?

La revisión del repositorio muestra una aplicación estructurada, con scripts de build, lint y pruebas E2E, pero desde esta revisión estática no es posible certificar que todas las funcionalidades funcionen correctamente. Aun si el sistema funcionara, eso no significa que esté bien desarrollado: la calidad también depende de legibilidad, modularidad, mantenibilidad, pruebas, documentación, seguridad y facilidad de cambio.